

1. Count words in a string using HashMap.

Simple Explanation: We split the input string into an array of words (using spaces as separators). Then, we use a **HashMap** where the key is the word, and the value is the count. We loop through the array, incrementing the count for each word we encounter.

```
import java.util.HashMap;
import java.util.Map;

public class WordCounter {
    public static Map<String, Integer> countWordFrequency(String text) {
        if (text == null || text.isEmpty()) {
            return new HashMap<>();
        }

        // Normalize and split the text into words
        String[] words = text.toLowerCase().split("\\s+");
        Map<String, Integer> wordCounts = new HashMap<>();

        // Count frequency of each word
        for (String word : words) {
            // Check if word is not empty after splitting
            if (!word.isEmpty()) {
                wordCounts.put(word, wordCounts.getOrDefault(word, 0) +
1);
            }
        }
        return wordCounts;
    }
}
```

2. Find the first non-repeating character in a string.

Simple Explanation: This is solved in two passes using a data structure that preserves insertion order (like a **LinkedHashMap**). In the first pass, we count the frequency of every character. In the second pass, we loop through the input again, and the first character whose count is exactly 1 is our answer.

```
import java.util.LinkedHashMap;
import java.util.Map;

public class NonRepeatingFinder {
    public static Character findFirstNonRepeating(String str) {
        if (str == null || str.isEmpty()) {
            return null;
        }

        // Use LinkedHashMap to preserve the order of characters
        // encountered
        Map<Character, Integer> counts = new LinkedHashMap<>();
```

```
// Pass 1: Count frequency
for (char c : str.toCharArray()) {
    counts.put(c, counts.getDefault(c, 0) + 1);
}

// Pass 2: Find the first character with a count of 1
for (Map.Entry<Character, Integer> entry : counts.entrySet()) {
    if (entry.getValue() == 1) {
        return entry.getKey();
    }
}
return null; // All characters repeat
}
```

3. Check if two strings are anagrams.

Simple Explanation: Two strings are anagrams if they have the exact same characters with the same counts, just in a different order (e.g., "listen" and "silent"). We check this by sorting the characters of both strings alphabetically. If the resulting sorted strings are equal, they are anagrams.

```
import java.util.Arrays;

public class AnagramChecker {
    public static boolean areAnagrams(String s1, String s2) {
        if (s1 == null || s2 == null || s1.length() != s2.length()) {
            return false;
        }

        // Convert to character arrays and sort them
        char[] chars1 = s1.toLowerCase().toCharArray();
        char[] chars2 = s2.toLowerCase().toCharArray();

        Arrays.sort(chars1);
        Arrays.sort(chars2);

        // Compare the sorted arrays
        return Arrays.equals(chars1, chars2);
    }
}
```

4. Compress a string using character counts (aabccc → a2b1c3).

Simple Explanation: We loop through the string, counting consecutive identical characters. When the character changes, we append the character and its count to the result. We repeat this until the end of the string.

```

public class StringCompressor {
    public static String compress(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }

        StringBuilder compressed = new StringBuilder();
        int count = 1;

        for (int i = 0; i < str.length(); i++) {
            // Check if the current character is the same as the next one
            if (i + 1 < str.length() && str.charAt(i) == str.charAt(i +
1)) {
                count++;
            } else {
                // Character changed or end of string reached
                compressed.append(str.charAt(i));
                compressed.append(count);
                count = 1; // Reset count for the next character
            }
        }
        // Return the original string if compression didn't save space
        (optional optimization)
        return compressed.length() < str.length() ? compressed.toString()
: str;
    }
}

```

5. Check if one string is a rotation of another string.

Simple Explanation: A string is a rotation of another (e.g., "ABCD" is a rotation of "CDAB") if the second string can be found inside a doubled version of the first string. We check by concatenating the first string to itself ("ABCDABCD") and checking if the second string ("CDAB") is a substring of the result.

```

public class RotationChecker {
    public static boolean isRotation(String s1, String s2) {
        if (s1 == null || s2 == null || s1.length() != s2.length() ||
s1.isEmpty()) {
            return false;
        }

        // Concatenate s1 with itself
        String s1s1 = s1 + s1;

        // Check if s2 is a substring of the doubled string
        return s1s1.contains(s2);
    }
}

```

6. Implement strstr() — find index of first occurrence of a substring.

Simple Explanation: The goal is to find the starting index of the first time the needle (substring) appears inside the haystack (main string). We use nested loops: the outer loop iterates through possible start positions in the haystack, and the inner loop compares characters of the needle at that position.

```
public class StrStrImpl {
    public static int findSubstring(String haystack, String needle) {
        if (needle == null || needle.isEmpty()) {
            return 0; // Empty needle is always found at index 0
        }
        if (haystack == null || haystack.length() < needle.length()) {
            return -1;
        }

        // Outer loop: iterate through possible starting positions in
        haystack
        for (int i = 0; i <= haystack.length() - needle.length(); i++) {
            int j;
            // Inner loop: compare characters of the needle
            for (j = 0; j < needle.length(); j++) {
                if (haystack.charAt(i + j) != needle.charAt(j)) {
                    break; // Mismatch found
                }
            }

            // If inner loop completed, we found a match
            if (j == needle.length()) {
                return i; // Return the starting index
            }
        }
        return -1; // Substring not found
    }
}
```

7. Convert Roman numerals to integer.

Simple Explanation: We process the Roman numeral string from **right to left**. We use a map to store the integer value for each symbol (I=1, V=5, etc.). If the current symbol's value is *less than* the previous symbol's value (e.g., 'IV' where I < V), we subtract it. Otherwise, we add it.

```
import java.util.HashMap;
import java.util.Map;

public class RomanToIntConverter {
    public static int romanToInt(String s) {
        Map<Character, Integer> values = new HashMap<>();
        values.put('I', 1);
        values.put('V', 5);
        values.put('X', 10);
    }
}
```

```

        values.put('L', 50);
        values.put('C', 100);
        values.put('D', 500);
        values.put('M', 1000);

        int result = 0;
        int prevValue = 0;

        // Process from right to left
        for (int i = s.length() - 1; i >= 0; i--) {
            int currentValue = values.get(s.charAt(i));

            if (currentValue < prevValue) {
                result -= currentValue; // Subtraction case (e.g., IV,
IX)
            } else {
                result += currentValue;
            }
            prevValue = currentValue;
        }
        return result;
    }
}

```

8. Convert integer to Roman numeral.

Simple Explanation: We create arrays to hold the standard Roman values (1000, 900, 500...) and their symbols ("M", "CM", "D"...). We loop through these values, repeatedly subtracting the largest possible value from the input integer and appending the corresponding symbol until the integer reaches zero.

```

public class IntToRomanConverter {
    public static String intToRoman(int num) {
        int[] values = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5,
4, 1};
        String[] symbols = {"M", "CM", "D", "CD", "C", "XC", "L", "XL",
"X", "IX", "V", "IV", "I"};

        StringBuilder roman = new StringBuilder();

        for (int i = 0; i < values.length && num > 0; i++) {
            // Repeatedly subtract the largest possible value
            while (num >= values[i]) {
                num -= values[i];
                roman.append(symbols[i]);
            }
        }
        return roman.toString();
    }
}

```

JAVA Interview Buddy